Module: realtime (rooms, chat, scheduler)

English version of ../../modules/realtime.md.

1. Purpose and boundaries

The Socket.IO layer: connecting players to rooms, round synchronisation, chat, and the scheduler that moves the game between stages. It does not contain scoring rules (that is the domain) and does not store room history in the database apart from the aggregate leaderboard record.

2. Use scenarios

- Joining a room by code, receiving its state.
- Starting a round by timer; broadcasting the question to every participant.
- Room chat; moderation by an admin.
- Failure: the room is full or does not exist → an error event.

3. Contract

Events of `app/realtime/socket_events.py` and `app/realtime/chat.py`; the HTTP part is `app/http_api/rooms.py`. The event list is fixed in ../api/conventions.md.

4. Data

Room state lives in process memory. Only the history of finished rooms is written to SQLite (`leaderboard.db`).

5. Access rules

The room host may control the round; an admin may moderate the chat (checked in `http_api/admin_routes.py`).

6. Dependencies

Outgoing: `game_logic`, `services.leaderboard`. Externally only the WebSocket transport. Incoming: the client `static/js/game.js`.

7. Configuration

`SOCKETIO_ASYNC_MODE` (`eventlet`/`threading`), `PORT`, `HOST`. The scheduler starts together with the application.

8. Build and debug

```
python -m pytest tests/test_realtime.py tests/test_socket_flow.py -q
```

9. Tests

`tests/test_realtime.py` (rooms and broadcasting), `tests/test_socket_flow.py` (a full round scenario), `tests/test_api.py` (the HTTP part of rooms).

10. Observability

Connection and room error logs; disconnects are the key signal when users report a "frozen game".

11. Limitations and readiness

- State in memory: a restart interrupts active rooms; several replicas are not supported.
- No player session recovery after a connection drop.
- Readiness: the round scenario is covered by a test; a dropped connection does not break other rooms.